

Graph Algorithms in Action

Modelling a Disease Outbreak

Take-Home Assignment Report

COSC2123/3119 — Algorithms and Analysis

RMIT University — Semester 1, 2026

Section	Marks
Task B — Infection Risk DP	4
Task C — Empirical Analysis	8
Task D — Antiviral Allocation	6
Total	18

Task A (adjacency matrix implementation) is code-only and carries no report marks per the assignment specification.

Task B — Infection Risk DP (Report, 1 page)

1. Pseudo-code (1 mark)

Algorithm TaskB(G, s, T)

Require: Graph $G = (V, E)$; source s ; horizon T

Ensure: Risk table where $\text{table}[t][i] = r_{\{t,i\}}$

```

1: table  $\leftarrow (T + 1) \times |V|$  array initialised to 0.0
2: table[0][s.index]  $\leftarrow 1.0$  // base case: only s is infected
3: for each day  $t = 1$  to  $T$  do
4:   prev  $\leftarrow$  table[t - 1]; curr  $\leftarrow$  table[t]
5:   for each vertex  $V_i \in V$  do
6:     escape  $\leftarrow 1 - \text{prev}[i]$ 
7:     for each neighbour  $V_j$  of  $V_i$  do
8:       escape  $\leftarrow$  escape  $\cdot (1 - \text{prev}[j] \cdot w_{\{ij\}})$ 
9:     curr[i]  $\leftarrow 1 - \text{escape}$ 
10:  curr[s.index]  $\leftarrow 1.0$  // pin patient zero (recurrence preserves it; defensive)
11: return table

```

2. Limitations of the recurrence (3 marks)

(a) Structural property. Intuitively, the recurrence double-counts certain infection scenarios because it treats each neighbour as an independent source of infection when in fact those neighbours often share the same upstream cause — or, in undirected graphs, are themselves infected only through V_i . Formally, the neighbour product treats the events “ V_j is infected by $t - 1$ ” as mutually independent across all $j \in N(V_i)$. That independence fails on any non-trivial graph through (i) shared upstream paths — different routes from the source to two neighbours of V_i share infection events that get double-counted — and (ii) undirected back-flow — a downstream neighbour V_j of V_i can only have been infected via V_i , yet appears in V_i ’s product as if it were an independent infector. Exact computation would need the full joint over $2|V|$ infection states; the recurrence trades that for polynomial cost by factorising into marginals.

(b) Figure 5, V_1 at $t = 3$. On the chain $V_0 - V_1 - V_2$ with $w_{01} = 0.8$, $w_{12} = 0.6$, the recurrence yields $r_{3,V_1} = 1 - (1 - 0.96)(1 - 1.0 \cdot 0.8)(1 - 0.48 \cdot 0.6) = 1 - 0.04 \cdot 0.2 \cdot 0.712 = 0.9943$ (the approximation the recurrence reports). The true risk is $1 - (1 - 0.8)^3 = 0.992$ (the exact probability that V_1 is infected by day 3, obtained from the closed-form single-edge cascade), because V_2 was infected only through V_1 , so any “back-transmission” $V_2 \rightarrow V_1$ happens only in scenarios where V_1 is already infected. The offending term $(1 - r_{2,V_2} \cdot w_{12}) = 0.712$ acts as if V_2 could infect V_1 via paths that don’t actually exist in the true process — V_2 ’s 0.48 probability mass was generated by V_1 at $t = 1$, so it cannot independently re-infect V_1 . This spuriously shrinks the escape product and so over-estimates the risk by 0.0023 absolute (0.9943 vs 0.992).

(c) Exact class. The recurrence is exact on stars centred at the source (every non-source vertex has V_0 as its sole neighbour; isolated vertices may also be present). On such a graph each leaf’s product collapses to a single factor $(1 - 1 \cdot w_{0i}) = (1 - w_{0i})$, and the recurrence unrolls to the exact closed form $r_{t,V_i} = 1 - (1 - w_{0i})^t$ — the probability of failing t independent daily transmission attempts. Both failure modes from (a) are absent: no cycle, and no back-flow (the only neighbour each leaf sees is the source itself, whose risk is fixed at 1). The class is strictly narrower than “trees”: (b) showed the recurrence is already approximate on the three-vertex path, which is itself a tree, so once any non-source vertex has degree > 1 back-flow becomes possible and the assumption breaks.

Task C — Empirical Analysis (Report, 2 pages)

Let $n = |V|$, $m = |E|$, $T = \text{horizon}$, $X = \text{MC simulations}$, $\deg(v) = \text{degree of } v$.

1. Algorithm Complexity Analysis (2 marks)

Algorithm × Representation	Complexity	Dominant operation
(a) MC × Matrix	$O(X \cdot T^2 \cdot n^2)$	row scan in <code>get_neighbours()</code> (line 6 Alg. 1) — n cells per call, ignoring zeros
(b) MC × List	$O(X \cdot T^2 \cdot (n + m))$	linked-list traversal — $\sum \deg(v) = 2m$ per inner day
(c) DP × Matrix	$O(T \cdot n^2)$	row scan in the neighbour-product loop (line 8 Alg. 2)
(d) DP × List	$O(T \cdot (n + m))$	linked-list traversal in the same loop

Monte Carlo pays an extra XT factor: X fresh simulations are launched at each of the T outer days, and each replays $t \in [1, t]$ days from scratch ($\sum t \approx T^2/2$ inner-day passes). The DP fills the table column-by-column reading only the previous column, so it visits each edge once per day rather than once per day-per-simulation. Swapping representations only changes the neighbour-loop cost: $O(n)$ per vertex for matrix versus $O(\deg(v))$ for list — lists dominate on sparse graphs, the gap closes at full density.

2. Empirical Design (2 marks)

Four free parameters drive the runtime: n , m (or density $d = m / C(n, 2)$), T , X . I vary one per plot and hold the others fixed so each plot isolates one asymptotic dimension. Other simulation parameters (`max_transmission_prob`, `vulnerability/dosage` ranges, graph seed) do not appear in the complexity, so they stay at default values.

Exp	Vary	Hold	What it isolates
(a)	$n \in \{10, 20, 40, 60, 80, 120\}$	$d = 0.30, T = 8, X = 50$	Scaling with population size
(b)	$T \in \{2, 5, 10, 20, 40\}$	$n = 60, d = 0.30, X = 50$	DP linear-in- T vs MC quadratic-in- T
(c)	$d \in \{0.05, 0.1, 0.2, 0.4, 0.8\}$	$n = 60, T = 8, X = 50$	Matrix density-independence vs list density-dependence
(d)	$X \in \{10, 30, 100, 300, 1000\}$	$n = 60, d = 0.30, T = 8$	MC linear-in- X ; DP independent

Each point is the mean of 3 trials on independent random graphs (graph seeds 100/101/102; MC noise seeds 7/8/9), timed with `utils/timer.py` inside the sweep driver `task_c_analysis.py`. Only the algorithm call is timed — graph construction is excluded per the spec tip. Plots use log axes when the swept parameter spans more than one decade so polynomial slopes appear as straight lines.

3. Empirical Analysis (2.5 marks)

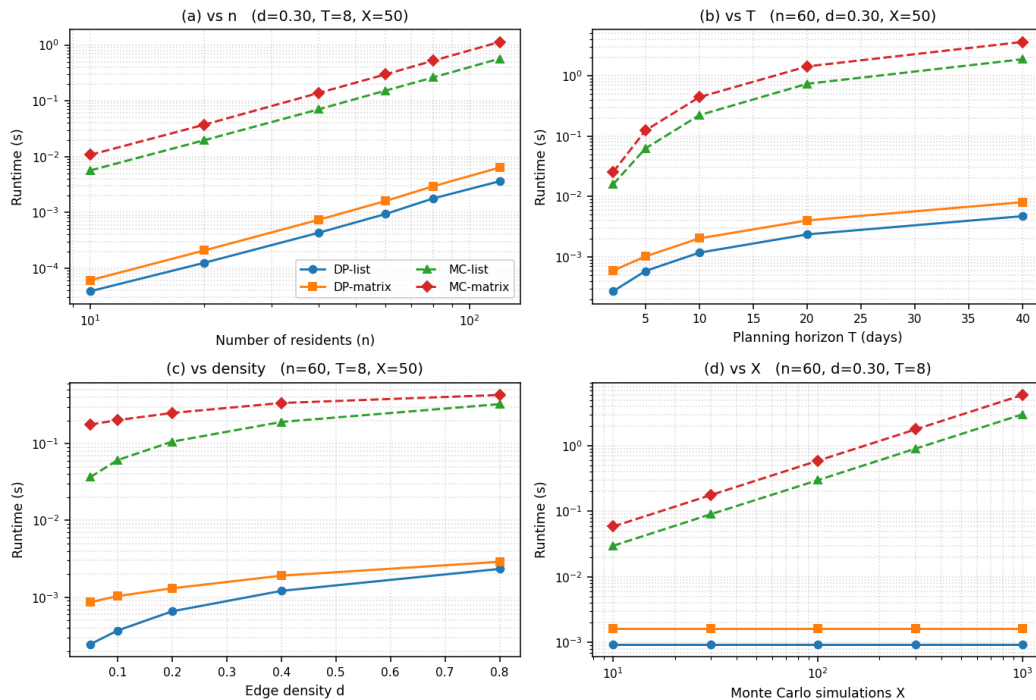


Figure 1: Runtime of all four (algorithm, representation) combinations across the four sweeps.

(a) vs n . Straight log-log lines of slope ≈ 2 across all four series — at $d = 0.30$, $m \approx 0.15n^2$ so even DP-list is dominated by an n^2 term. DP and MC clusters are separated by $\sim 150\times$ (within an order of magnitude of the predicted $XT = 400$). Within each cluster matrix is $\sim 1.7\times$ slower than list, smaller than the naive $1/d \approx 3.3\times$ because the matrix's contiguous row loop benefits from CPU cache locality.

(b) vs T . DP grows linearly ($T \times 4 \Rightarrow \text{time} \times 4$). MC bends upward: MC-list grows $222 \rightarrow 1878$ ms over $T = 10 \rightarrow 40$ ($\times 8.4$), close to the predicted $T^2 = \times 16$, slightly sub-quadratic due to fixed per-day overhead. The T^2 blow-up dominates regardless of representation.

(c) vs density. Matrix curves are nearly flat (the row scan visits n cells regardless of zeros); list curves rise linearly with m . List-matrix gap is $\sim 5\times$ at $d = 0.05$ and shrinks to $\sim 1.3\times$ at $d = 0.80$ — the textbook crossover where $\sum \deg(v)$ approaches n^2 .

(d) vs X . Both MC variants are slope-1 log-log lines (linear in X); DP lines are flat at ~ 0.9 ms (list) / ~ 1.7 ms (matrix). At $X = 1000$ the MC error on cyclic networks is still 1–2 % per cell vs the exact DP, so X is rarely cut below the hundreds — locking in a $100\times$ slowdown at any useful accuracy.

4. Reflection (1.5 marks)

Results match §1: every slope agrees with the dominant complexity term within constant factors, and the qualitative features (matrix density-independence, MC quadratic in T , DP independent of X) are visible at a glance. Minor deviations: MC is slightly sub-quadratic in T due to fixed per-day RNG/bookkeeping overhead; at small n DP-matrix is only marginally slower than DP-list because contiguous rows are more cache-friendly than linked lists.

Recommendation for Metropolis. The city is large, sparsely connected, modest T . Cost ordering: DP-list ($O(Tm)$) $\ll$ DP-matrix ($O(Tn^2)$) $\ll$ MC-list ($O(XT^2m)$) $\ll$ MC-matrix ($O(XT^2n^2)$). DP-list wins on every axis — exact, X -free, linear in T , scales with m not n^2 . At $n = 60$, $d = 0.05$ it ran $\sim 145\times$ faster than MC-list (sub-second per day vs minutes at city scale) and $\sim 3.5\times$ faster than DP-matrix. Daily updates to weights/connections do not change this: the DP recurrence is structural, generalises to time-varying $w_{ij,t}$ at the same $O(Tm)$ cost, and is a sub-second per-day job at city scale; MC gains nothing from incremental updates either. MC only overtakes DP when the model itself outgrows a closed-form recurrence (heterogeneous incubation, agent memory) and exact computation becomes intractable.

Task D — Antiviral Allocation (Report, 2 pages)

Let n = eligible residents, C = total dose budget, c_i = dose requirement, b_i = infection-risk benefit.

1. Algorithm Design (1 mark)

0/1 knapsack with real-valued benefits and a fewest-doses tiebreak, solved by top-down memoisation (`solve()` in `treatment/task_d.py`). A 2-D memo of size $(n + 1) \times (C + 1)$ stores per cell `memo[i][c] = (best_benefit, min_doses_to_achieve_best_benefit)` — the tuple captures the tiebreak inside the cell so no second pass is needed.

Base cases. `solve(0, c) = solve(i, 0) = (0.0, 0)`.

Recurrence. For $i \geq 1, c \geq 1$:

```
skip = solve(i - 1, c)                # don't vaccinate person i
take = solve(i - 1, c - c_i) + (b_i, c_i) # if c_i ≤ c
memo[i][c] = take if take.benefit > skip.benefit (strictly better)
                OR (take.benefit == skip.benefit AND take.doses < skip.doses)
                else skip
```

Plain lexicographic max of (benefit, -doses). Float tolerance $1e-12$ — benefits are probabilities in $[0, 1]$ with sums below n , well above double-precision rounding noise.

Top-down (not bottom-up): memoising only the cells reached from (n, C) is far fewer than nC on instances with large per-person costs (Case 6 test rewards this).

Backtracking. Walk i from n down to 1, replaying the same skip-vs-take comparison through `solve()` (not raw memo lookups) and deducting c_i from a running capacity whenever take wins. Routing through `solve()` keeps the $i = 0 / c = 0$ base cases uniform without indexing into unmemoised cells.

2. Complexity (1 mark)

Time $O(nC)$, space $O(nC)$. At most nC unique (i, c) cells, each $O(1)$ work plus two memoised recursive calls. Backtracking adds $O(n)$; recursion stack adds $O(n)$. Pseudo-polynomial: linear in C but independent of how large individual c_i are. Exponentially faster than brute force's $O(2^n)$ whenever $C < 2n/n$ — always true for realistic intervention budgets.

3. Extension 1 — Triage by K vulnerability tiers (2 marks)

Partition residents into tiers T_1 (most vulnerable) ... T_K (least). T_k must be allocated optimally before any dose is spent on T_{k+1} .

Modification. Run the same DP K times in cascade, threading the remaining capacity through:

```
remaining = C; selected = []
for k = 1 to K do
    subset_k, _, used_k, _ = task_d(T_k, remaining)
    selected += subset_k; remaining -= used_k
return selected
```

Complexity. Each call is $O(|T_k| \cdot C)$ in the worst case. Summing: $O((|T_1| + \dots + |T_K|) \cdot C) = O(nC)$ — same class as the un-triaged DP (the extra K is hidden in constants because per-tier capacity strictly decreases).

Counterexample — why ignoring vulnerability hurts. $C = 4$ with three residents:

Resident	Tier	b	c
VA	1 (highly vulnerable — would die if infected)	0.50	4
VB	2 (low vulnerability)	0.60	2
VC	2 (low vulnerability)	0.55	2

Plain knapsack picks {VB, VC}: benefit 1.15, 4 doses, VA left unprotected. Triage DP picks VA first (only feasible tier-1 choice): benefit 0.50, 4 doses, tier 2 receives nothing. The plain knapsack has higher aggregate rT-benefit yet leaves the only resident who would die exposed — triage restores clinical priority at the cost of some aggregate risk-reduction, exactly the trade-off the model should make explicit.

4. Extension 2 — Interdependent vaccinations (2 marks)

Why independence is required for the DP. The DP's optimal substructure relies on the marginal contribution of vaccinating i being a constant b_i , regardless of who else is in the subset. That is what lets $\text{memo}[i][c]$ be a function of (i, c) alone. If b_i depends on who else was vaccinated, different partial solutions reaching the same (i, c) state would credit person i with different benefits later, so the "best" cell value depends on history rather than state.

Counterexample. Contact chain $V0 \rightarrow V1 \rightarrow V2$ with high weights and $r_{1,T} = r_{2,T} = 0.95$. Assume vaccinating a resident drops every neighbour's rT to ≈ 0 (the neighbour's only exposure pathway is blocked). Costs $c_{V1} = c_{V2} = 1$, budget $C = 1$.

- DP sees benefits (0.95, 0.95) and costs (1, 1) — either choice equal $\rightarrow$ reports benefit 0.95.
- True benefit of vaccinating V1: V1 protected (0.95) and V2's residual risk $\rightarrow 0$ (saving another 0.95) $\rightarrow$ true total ≈ 1.90 .
- True benefit of vaccinating V2: V2 alone protected; V1 stays at 0.95 $\rightarrow$ true total ≈ 0.95 .

Vaccinating V1 is twice as good as V2, yet the independent-benefit DP treats them as tied because it cannot see the herd-effect interaction.

Why the true problem is significantly harder. The benefit becomes a set function $B(S)$ — each evaluation requires re-running the Task B risk DP on the residual graph induced by S . Three consequences:

(i) $\text{memo}[i][c]$ no longer captures enough state — optimal substructure would need to encode the entire vaccinated subset so far (2^n states); (ii) per-evaluation cost balloons (one Task B re-run, $O(T_m)$ each), giving brute force $O(2^n \cdot T_m)$; (iii) the resulting problem matches a well-studied template in the optimisation literature that lies outside this course's syllabus — I'll spell out the connection in plain terms before naming it.

From this assignment to the standard problem. Define the ground set $U =$ the n eligible residents, a set function $B : 2^U \rightarrow \mathbb{R}$ where $B(S) =$ total expected risk reduction across the population when exactly the residents in S are vaccinated (one Task B DP run on the residual graph), and the knapsack constraint $\sum_{i \in S} c_i \leq C$ (the dose budget). Under the independent-cascade transmission model used here, B has two properties:

- Monotone: $B(S \cup \{v\}) \geq B(S)$ — adding someone to the vaccinated set never increases anyone else's residual risk, so vaccinating one more person can never hurt.
- Submodular ("diminishing returns"): the marginal value $B(S \cup \{v\}) - B(S)$ shrinks as S grows. Intuitively, the more of v 's neighbours are already protected, the less additional protection vaccinating v contributes — much of the herd-effect benefit is already provided by S .

Maximising a monotone submodular function under a knapsack constraint is a standard problem in combinatorial optimisation. I rely on two imported results (these are not derived in this report — they are established facts I'm pointing to): the problem is NP-hard, by reduction from set cover via the influence-maximisation framing of Kempe, Kleinberg & Tardos (KDD 2003); and a cost-modified greedy algorithm achieves a tight $(1 - 1/e) \approx 0.632$ approximation guarantee (Sviridenko, Operations Research Letters, 2004; building on Nemhauser, Wolsey & Fisher's 1978 result for the uniform-cost case). External notes on submodular optimisation by Krause & Golovin (2014) helped me understand the framing; the assignment itself does not require this background.

Practical takeaway. Exact solution requires exponential time unless $P = NP$, but a simple cost-modified greedy gets within $\sim 37\%$ of optimal — and the independent-benefit knapsack I implemented is a deliberate relaxation of this harder problem: cheap and exact within its abstraction (treating each b_i as fixed), but blind to the herd-effect amplification that the counterexample above demonstrates.